Faculty of Engineering and Technology

Department of CSE – AI, ML and DS

PROJECT REPORT

Prompt Book and Development Record

Semester 3 – CSE (AI/ML)

Project Title

The Butterfly Effect

Submitted by

Student name	Raj A. Khakhkhar
GR number / Roll number	92610122002
Division	3EI1
College / institute	Marwadi University
University	Marwadi University
Faculty / guide	Chandreshkumar Vyas
Academic year / semester	2026 / Semester 3 – CSE (AI/ML)

Faculty of Engineering and Technology | Department of CSE – AI, ML and DS | Semester 3 – CSE (AI/ML)

Prepared 30 October 2026

Record scope

This prompt book reproduces the three user prompts available in the supplied Codex attachment files. It distinguishes those texts from summaries of outcomes. The initial project handoff says development previously used OpenCode, but no OpenCode transcript or original prompt log was available; those prompts are not reconstructed or invented.

Development record

This book includes three actual Codex user prompts supplied in text form. They are reproduced faithfully in the following sections; screenshots of prompt conversations were not supplied.

Stage 1 — Idea and problem analysis: Prompt 1 establishes the scenario and asks for inspection of the existing application. Earlier OpenCode prompt evidence is unavailable.

Stage 2 — Requirements and features: Prompts 1 and 2 specify controls, calculations, presets, baseline comparison, reset, and evidence rules.

Stage 3 — UI and layout design: Prompt 2 describes visual hierarchy, responsive layout, interaction clarity, and restrained presentation. A separate historical UI prompt is not available.

Stage 4 — Code generation and implementation: Prompts 1 and 2 direct targeted changes to the existing React/TypeScript application while preserving the project.

Stage 5 — Testing and verification: Prompts 1 and 2 request builds, lint, automated tests, and browser interactions; documented results and limitations are summarized below.

Stage 6 — Debugging and improvement: Recorded outcomes cover input validation, zero-budget handling, model wording, local font fallbacks, and tests. Original debugging follow-up prompts are unavailable.

Prompt screenshot status: no genuine prompt screenshots were supplied. Application screenshots belong in the report only and are not substituted here.

Prompt 1 — diagnosis and repair

AI tool: Codex desktop assistant. The project handoff reports an earlier OpenCode phase, but this exact prompt was supplied for Codex work. Purpose: The Butterfly Effect project handoff requesting inspection, repair, verification, and a running preview.

Intended outcome: follow the request below while preserving the existing application and reporting evidence accurately.

You are taking over an existing AI-assisted web application project called **The Butterfly Effect**.

PROJECT LOCATION: `D:\butterfly-effect`

YOUR MISSION

Inspect the ENTIRE existing project, understand what has already been built, diagnose all errors, fix them, and get the actual application running so I can see and interact with it.

Do not start over. Do not replace the project with a generic template. Preserve all useful existing work and improve it carefully.

I have limited time and will use only two major prompts with you. This first prompt is for complete diagnosis, repair, and verification. The second prompt will handle the final improvements and remaining submission requirements.

CONTEXT

The project was previously developed using OpenCode. The intended application includes:

- A premium, dark-themed, interactive Butterfly Effect experience.
- Scenario variables controlled through sliders.
- A deterministic cause-and-effect simulation engine.
- A visual graph showing how changes propagate through connected variables.
- Results showing expenditure, budget remaining or deficit, travel impact, schedule risk, and readiness.
- A baseline-versus-current scenario comparison.
- Preset experiments such as doubling guests, choosing an expensive venue, reducing preparation time, and increasing

the budget.
- Reset functionality.

The project may contain incomplete code, duplicated declarations, malformed JSX, missing imports, inconsistent variable identifiers, incorrect React hooks, dependency problems, or other errors. These are possibilities based on previous logs; inspect the actual current files to determine what is genuinely wrong.

Previously, `npm install` succeeded and installed 28 packages. Vite reported that it started at `http://localhost:5173/`, but the application has not been verified. A previous build was not confirmed. There was also an earlier disk-space error, so avoid unnecessary installations.

PHASE 1 — INSPECT BEFORE CHANGING ANYTHING

1. Confirm that the project folder exists and inspect its directory structure.
2. Read `package.json`, the entry HTML, all relevant React/TypeScript files, CSS files, simulation logic, configuration files, and existing assets.
3. Identify the actual framework, installed dependencies, current implementation, and existing features.
4. Check available disk space if possible.
5. Identify all genuine build, type, runtime, and logic errors.
6. Check whether a development server is already running before starting another.
7. Do not assume that previous agent notes accurately describe the current files.

PHASE 2 — FIX THE PROJECT

Repair the existing project with minimal unnecessary changes.

Requirements:

- Resolve all build-blocking TypeScript, JSX, import, and configuration errors.
- Fix broken React state management, hooks, and rendering.
- Fix simulation calculations and inconsistent dependency/node identifiers.
- Remove accidental duplicate declarations and duplicated UI sections.
- Ensure sliders update the simulation correctly.
- Ensure results recalculate when variables change.
- Ensure the baseline comparison is meaningful and stable.
- Ensure preset experiments actually change the relevant variables.
- Ensure reset restores the intended initial scenario.
- Ensure the cause-and-effect visualization reflects the simulation data.
- Make all visible controls work. Do not leave fake buttons or decorative controls that imply functionality they do not have.
- Preserve the existing visual direction and make sensible improvements where needed.
- Ensure the layout is usable on desktop and mobile.
- Avoid unnecessary packages, backend services, paid APIs, and API keys.
- Do not delete useful assets or rewrite everything just to make the code easier for you.

PHASE 3 — BUILD AND TEST

Do not stop after editing files.

1. Install dependencies only if genuinely necessary.
2. Run the project's actual build command.
3. Fix build errors and rerun the build until it succeeds.
4. Run available linting and type checks where practical.
5. Test the simulation calculations and important interactions.
6. If browser automation or a browser preview is available, open the application and test the real rendered page.
7. Check for runtime errors and broken assets.
8. If a browser is unavailable, clearly distinguish automated checks from actual browser testing.
9. Start the development server only if needed. Avoid creating duplicate servers.
10. Provide the exact local preview address and clear instructions for opening it.

A successful server startup alone is NOT proof that the application works. A successful build alone is NOT proof that all features work.

IMPORTANT ENVIRONMENT RULES

- The project is on Windows.
- Use commands compatible with the actual available shell.
- Do not assume Bash commands work in PowerShell.
- Do not run destructive cleanup commands.

- Do not delete project files, caches, or dependencies without a demonstrated need.
- Do not move the project or change its location.
- Do not overwrite unrelated projects.
- Never claim a test passed unless you actually ran it and observed the result.

FINAL RESPONSE

When finished, report:

1. What was wrong.
2. What you changed.
3. Whether the production build succeeds.
4. Which features you actually tested.
5. Which tests passed and failed.
6. Whether you personally verified the rendered application in a browser.
7. The exact local preview address.
8. Any remaining issues or limitations.

Prioritize a working, demonstrable application over adding more features.

Start by inspecting the existing project. Then complete the repair and verification process rather than merely proposing a plan.

Recorded outcome and verification

The application was inspected and repaired rather than replaced. The later project record shows successful build and lint, browser testing of all presets, reset, and slider interaction, and a visible local preview. The detailed repair history predates this report task and is summarized from the project handover and current conversation record.

Prompt 2 — final development pass

AI tool: Codex desktop assistant. The project handoff reports an earlier OpenCode phase, but this exact prompt was supplied for Codex work. Purpose: The final application quality, testing, deployment-readiness, and handover request.

Intended outcome: follow the request below while preserving the existing application and reporting evidence accurately.

We are continuing the existing IPE project, **The Butterfly Effect**, located at `D:\butterfly-effect`.

The application already builds and runs at `http://127.0.0.1:5173/`. The previous Codex task repaired the application, and the reported checks were:
- Production build succeeded.
- Lint passed without warnings.
- The rendered page was checked in a browser.
- All four presets, reset, and a budget slider were tested successfully.
- Browser console logs were not inspected.
- The C: drive is full, but D: has available space.

This is the FINAL major development pass. Work carefully and preserve the functioning application.

1. Inspect the current implementation

Before making changes:
- Read the existing source code and project configuration.
- Check the current UI and implemented functionality.
- Identify unfinished features, weaknesses, and actual bugs.
- Do not assume anything is broken without checking.
- Do not replace the app with a new template or rebuild it from scratch.

2. Improve the presentation

The goal is a polished, memorable application suitable for a university end-semester demonstration.

Preserve the existing visual identity and improve it where genuinely beneficial:
- Strong visual hierarchy and polished typography.
- Consistent spacing, alignment, and component styling.
- Premium dark interface with thoughtful use of color and contrast.
- Clear visual distinction between inputs, simulation results, and cause-and-effect relationships.

- Smooth, restrained transitions and useful hover/focus states.
- Responsive layout for common desktop and mobile screen sizes.
- Clear empty, loading, success, and error states wherever relevant.
- Accessible labels, readable text, keyboard usability, and visible focus indicators.

Avoid excessive animation, unnecessary decoration, fake 3D effects, and visual clutter. Do not sacrifice clarity or functionality for appearance.

3. Verify the actual functionality

Inspect and test the following:

A. Scenario controls
- All sliders accept valid values.
- Displayed values match the actual simulation inputs.
- Changing an input recalculates dependent outputs correctly.
- Boundary values do not cause invalid calculations.

B. Simulation engine
- Calculations are deterministic and internally consistent.
- Budget remaining or deficit is calculated correctly.
- Risk/readiness indicators follow documented, sensible rules.
- Related values update when their dependencies change.
- Invalid values cannot silently corrupt the results.

C. Cause-and-effect visualization
- Nodes and connections correspond to real simulation dependencies.
- Selecting a node or path provides meaningful tracing where implemented.
- The graph remains legible and usable at supported screen sizes.
- Do not claim that the app models real-world causality scientifically. Clearly treat its rules as a simplified, illustrative simulation.

D. Baseline comparison
- The baseline remains stable while the current scenario changes.
- Differences are calculated and displayed correctly.
- Reset restores the intended initial state.

E. Presets and reset
- Test all four existing presets.
- Confirm each preset changes the intended inputs and updates the results.
- Confirm reset restores the default scenario and baseline comparison.

F. Runtime and production
- Check for console errors if browser tooling permits.
- Check for broken assets and runtime errors.
- Run the production build and lint/type checks available in the project.
- Fix verified issues and rerun failed checks.
- Test the production build locally if practical.
- Never claim that something was tested unless you actually tested it.

Add or improve lightweight automated tests for core calculations if the existing setup supports this without substantial dependencies or disruption. Do not introduce a large testing framework just for appearances.

4. Preserve the working environment

- Keep the project at `D:\butterfly-effect`.
- Do not touch the separate `D:\mystery-engine` project.
- Do not delete project files, caches, or dependencies without a clear reason.
- Avoid unnecessary package installations because the C: drive is full.
- If temporary files must be created, prefer a suitable location on D:.
- Do not expose credentials, API keys, or private information.
- Do not add a backend or paid service unless absolutely necessary.
- Keep the application usable without API keys.

5. Prepare for deployment

Inspect the project and determine the simplest suitable way to publish the existing frontend as a public website.

- Confirm the production build output and required assets.
- Check whether the project is ready for static hosting.
- Identify any hard-coded localhost URLs, missing assets, or routing issues that would prevent deployment.
- Make only necessary, safe code/configuration changes to prepare for deployment.
- Do not pretend that deployment has happened if it has not.
- Do not create accounts, accept terms, publish data, or perform external actions without my explicit approval.
- Explain the simplest deployment route, any account requirements, and the exact steps I must perform to obtain a public URL.

6. Prepare evidence for the IPE report

Report the actual implemented functionality and verified test results in a structured format that can later be used in my university report and prompt book.

Include:
- Project purpose and the problem it addresses.
- Main features actually implemented.
- Technology stack confirmed from the project.
- Actual changes made during this task.
- Tests performed, expected outcomes, observed outcomes, and pass/fail status.
- Known limitations and remaining tasks.
- Deployment readiness and next steps.

Do not invent prompts, test results, screenshots, user feedback, or completed features. Do not write a fictional development history. Preserve the distinction between work performed before this task and work performed during this task.

If helpful, create a concise project handover document inside the project folder, but do not generate a final university PDF with invented personal details or fabricated evidence.

7. Final response

At completion, clearly state:

1. What you improved.
2. What you tested and the exact results.
3. Whether production build and lint/type checks passed.
4. Whether browser console inspection was possible.
5. Whether the application is ready for deployment.
6. The exact command to run it locally.
7. The remaining steps needed to publish a public link.
8. The files changed or created.
9. Any unresolved errors or limitations.

Prioritize correctness, reliability, and a polished demonstration over unnecessary new features.

Begin by inspecting the current project. Complete the work, verify it, and provide the final report. Do not stop after producing a plan.

Recorded outcome and verification

The final development pass added finite/non-negative validation, zero-budget gauge behavior, an explicit illustrative-model description, local font fallbacks, and six Node-native tests. The observed final checks were: production build exit 0; Oxlint exit 0 with no diagnostics; six tests passed and zero failed; production preview at 127.0.0.1:4173 rendered and its double-guests preset and reset were exercised.

Limitations in the record: no browser console inspection, no full graph keyboard node-selection test, and no public deployment.

Prompt 3 — report and prompt book

AI tool: Codex desktop assistant. The project handoff reports an earlier OpenCode phase, but this exact prompt was supplied for Codex work. Purpose: The current request to inspect the university format, prepare the report and prompt book, and list student actions.

Intended outcome: follow the request below while preserving the existing application and reporting evidence accurately.

We are completing the university IPE project titled **The Butterfly Effect**.

Project directory: `D:\butterfly-effect`

Your task is to prepare the complete project report and the required supporting documentation according to the official university report format PDF provided in the project directory.

I want you to do as much of the work as possible automatically. I will personally complete any tasks that genuinely require my information, manual actions, or screenshots.

IMPORTANT: Do not stop after proposing an outline. Inspect the available materials, prepare the documents, and report exactly what remains for me to do.

PHASE 1 — FIND AND STUDY THE UNIVERSITY FORMAT

1. Inspect the project directory and identify the university-provided report-format PDF.

2. Read the entire PDF, including its headings, instructions, page order, formatting requirements, tables, declarations, and required sections.

3. If the PDF contains scanned pages or images, inspect those pages as necessary.

4. Treat the university's format as the authority. Follow its structure, terminology, and requirements as closely as possible.

5. Do not replace the university's format with a generic project report.

6. If the format file is missing, unreadable, or ambiguous, explain the problem and do not pretend you have read it. Complete any independent preparation that is still possible.

PHASE 2 — INSPECT THE ACTUAL PROJECT

Inspect the existing application and its documentation, including:

- `PROJECT_HANDOVER.md`, if present.

- `package.json`.

- The simulation implementation and tests.

- The React application and relevant styles.

- Existing project documentation and available screenshots.

- Any records of the prompts used during development.

Understand what the application actually does before writing about it.

The application is called **The Butterfly Effect**. It is an interactive web application that demonstrates how changing variables in a scenario can affect connected variables and outcomes.

The existing implementation includes scenario sliders, a simulation engine, a cause-and-effect visualization, result calculations, a baseline comparison, presets, and reset functionality.

Previously reported verification results include:

- Production build succeeded.

- Oxlint passed without warnings.

- Six simulation tests passed.

- Browser testing confirmed that several presets, sliders, and reset functionality worked.

- Some browser checks remain incomplete, including browser console inspection and complete verification of keyboard-based graph selection.

- The application has not yet been deployed publicly.

Verify these claims against the available project files and test records wherever possible. Distinguish previously reported results from checks you perform yourself. Do not claim that a feature or test has been verified if you cannot establish that it was.

PHASE 3 — PREPARE THE UNIVERSITY REPORT

Create the complete report using the university's required structure.

Populate every section you can complete reliably from the project files, the actual development history, the available prompts, and verified test results.

Where applicable, cover:

- Project title and overview.
- Problem statement and motivation.
- Objectives.
- Proposed solution and intended users.
- Technology stack.
- Application architecture and workflow.
- Main features and functionality.
- AI tools used and how they contributed.
- Prompt engineering and development methodology.
- Testing methodology and actual results.
- Screenshots or other evidence required by the university.
- Results, limitations, and future improvements.
- Conclusion.
- References, declarations, acknowledgements, or other sections required by the university format.

These are possible content areas, not a replacement for the university's actual required sections.

Write clear, professional, academically appropriate content. Explain technical concepts accurately without unnecessary jargon.

Describe the application as an illustrative simulation of cause-and-effect relationships. Do not claim it scientifically predicts every real-world consequence.

Use the actual technology stack identified in the project files.

Do not invent research, statistics, references, user surveys, performance measurements, personal contributions, or experimental results.

PHASE 4 — HANDLE MY PERSONAL DETAILS

I will supply personal information such as:

- Student name.
- Roll number or GR number.
- Division.
- College or university details, if required.
- Faculty or guide name, if required.
- Other identifying information explicitly required by the university.

Do not invent these details.

Where information is missing, use highly visible placeholders such as:

[STUDENT ACTION REQUIRED: Enter your full name.]

[STUDENT ACTION REQUIRED: Enter your GR number.]

[STUDENT ACTION REQUIRED: Enter your division.]

Use the exact field names required by the university format.

Do not repeatedly interrupt the work to ask me questions. Finish the portions you can complete, then provide one consolidated list of missing information.

PHASE 5 — HANDLE SCREENSHOTS AND MANUAL EVIDENCE

Inspect whether suitable screenshots already exist.

Use existing genuine screenshots only when they accurately represent the current application. If you have access to a working browser and can capture screenshots of the actual application, do so where appropriate.

Never create fake screenshots, fabricate browser results, or use a mockup as evidence that a feature works.

If a screenshot must be captured manually, insert a clearly marked placeholder in the document, for example:

[STUDENT ACTION REQUIRED: Insert a screenshot of the actual application dashboard. Open the application in your browser, ensure the dashboard is visible, capture a screenshot, and insert it here.]

For each required screenshot, specify:

1. What must be visible.

2. How I should produce it.

3. Which section or page it belongs in.

Keep screenshot placeholders visually obvious so I cannot accidentally submit them as completed evidence.

Do not leave vague instructions such as "add screenshots later." Tell me precisely which screenshots are needed and what each should demonstrate.

PHASE 6 — PREPARE THE PROMPT BOOK

If the university requires a prompt book or prompt-engineering record, prepare it as a separate document unless the university explicitly requires it inside the main report.

Use the actual prompts available from the development history and project records.

For each prompt, record:

- Prompt number and purpose.

- The actual prompt text, reproduced faithfully where available.

- The AI tool used.

- The intended outcome.

- The actual outcome or changes, supported by available evidence.

- Tests or verification performed afterward.

- Any limitations or follow-up actions.

Include the development process from initial generation through debugging and final improvements, using the available records.

Do not invent missing prompts or pretend you have access to conversations or logs that are unavailable. If a prompt cannot be recovered, clearly mark it as unavailable and tell me whether I need to provide it.

The record must reflect the real development process, including the use of OpenCode and the subsequent switch to Codex.

PHASE 7 — CREATE THE DOCUMENTS

Create the deliverables in the project directory, preferably in a dedicated `report` folder.

Produce:

1. The completed main project report.

2. The prompt book, if required as a separate document.

3. Any additional document explicitly required by the university format.

Use an editable DOCX file for documents when appropriate, together with a PDF version when feasible.

Preserve the university's specified page order, headings, tables, margins, title-page structure, and other formatting requirements as closely as possible.

Ensure the final PDF is readable and properly formatted. Check page breaks, tables, headings, blank pages, image placement, and any remaining placeholders.

If you cannot reproduce a specific formatting element or generate a PDF, explain the limitation and provide the best usable alternative. Do not claim a document is complete if it still contains unfinished required fields or screenshot

placeholders.

Use this filename pattern for the final PDF if the university requires it:

`Division_GR No_Name.pdf`

Do not invent the filename details. Use placeholders until I provide them.

PHASE 8 — FINAL MANUAL-ACTION CHECKLIST

When you finish, give me a separate, simple checklist titled:

WHAT I MUST DO MYSELF BEFORE SUBMISSION

List every action I must perform, including any applicable:

- Personal details I must provide.
- Screenshots I must capture and where to insert them.
- Missing prompts or records I must supply.
- Manual formatting or document corrections.
- Final checks I must complete.
- Deployment or public-link requirements.
- PDF filename changes.
- Submission-form fields or uploads required by the university.

For each action, tell me exactly what to do and where to do it.

Separate these categories clearly:

A. Tasks you completed.

B. Tasks you could not complete because information or access was missing.

C. Tasks I must personally perform.

D. Final checks required before submission.

If an action is unnecessary, do not add it merely to make the checklist longer.

STRICT RULES

- Never modify or damage the working application unnecessarily.
- Do not overwrite existing project files without checking their contents first.
- Do not fabricate student information, screenshots, prompt history, references, or test results.
- Do not claim the application is publicly deployed unless it actually is.
- Keep all project documents inside `D:\butterfly-effect`, preferably in a dedicated report folder.
- Preserve the original university format.
- Be honest about every limitation.

Start by locating and reading the university PDF. Then inspect the project and prepare the documents. Continue until you have produced everything you can complete reliably, and finish with the exact manual-action checklist.

Recorded outcome and verification

The written university-format requirements supplied for this audit define the seven main sections, cover wording, and evidence rules; the revised report follows them. The low-resolution cover example does not confirm additional graphical details. Genuine prompt screenshots and the original OpenCode prompt history were not supplied, and the application has not been publicly deployed.

Prompt history that remains unavailable

The source project handoff identifies OpenCode as an earlier development tool, but no original OpenCode prompt text, conversation transcript, or tool logs were found in the inspected project files or supplied attachments. The prompts above are the available user instructions for the three Codex work phases. The student should provide any original OpenCode records if the university requires them.

No.	Purpose and tool	Outcome supported by available record
1	Diagnose, repair, verify app — Codex; handoff notes an earlier OpenCode phase.	Earlier development record reports successful build/lint, browser checks of presets, reset, and slider interaction, plus a local preview. Historical result, not rerun during this document audit. Original OpenCode prompts unavailable.
2	Final application pass — Codex.	Previously recorded final run: finite/non-negative validation, zero-budget handling, illustrative-model wording, local font fallbacks, and six Node tests. Production build and Oxlint passed; six tests passed; production preview and double-guests/reset were observed. Not rerun during this document audit.
3	Report and prompt-book preparation — Codex.	This audit corrected the report and prompt book using the written seven-section format supplied by the student. Three actual Codex prompt texts are retained verbatim. No prompt screenshots or public deployment were added.